complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + reduce() aggregate (sum, max, concat)

filter() null checks + reduce() aggregate (sum, max, concat)

filter() comparisons + reduce() aggregate (sum, max, concat)

filter() method calls returning boolean + reduce() aggregate (sum, max, concat)

distinct() rely on equals() / hashcode() + reduce() aggregate (sum, max, concat)

distinct() immutable objects + reduce() aggregate (sum, max, concat)

map() transform objects + reduce() aggregate (sum, max, concat)

map() create new objects + reduce() aggregate (sum, max, concat)

map() call pure functions + reduce() aggregate (sum, max, concat)

flatmap() return stream + reduce() aggregate (sum, max, concat)

flatmap() split data + reduce() aggregate (sum, max, concat)

flatmap() expand collections + reduce() aggregate (sum, max, concat)

sorted() natural ordering + reduce() aggregate (sum, max, concat)

sorted() custom comparator + reduce() aggregate (sum, max, concat)

sorted() multi-field compare + reduce() aggregate (sum, max, concat)

peek() logging + reduce() aggregate (sum, max, concat)

peek() debugging + reduce() aggregate (sum, max, concat)

peek() metrics + reduce() aggregate (sum, max, concat)

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where the **terminal operation is reduce()** and it demonstrates **aggregation**:

- **SUM** (numeric aggregation)
- **MAX** (numeric aggregation)
- **CONCAT** (string aggregation)

---

1 filter() boolean checks + reduce() Aggregate

```
import java.util.*;

public class FilterBooleanReduceAggregate {
   public static void main(String[] args) {

      List<Integer> nums = new ArrayList<Integer>();
      nums.add(10);
      nums.add(15);
      nums.add(20);

      int sum = 0;
      int max = 0;
      String concat = "";

      for (Integer n : nums) {
         if (n % 2 == 0) {
            sum += n;
            if (n > max) {
               max = n;
            }
            concat += String.valueOf(n);
         }
      }

      System.out.println(sum + " | " + max + " | " + concat);
   }
}
```

---

2 filter() null checks + reduce() Aggregate

```
import java.util.*;

public class FilterNullReduceAggregate {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
      names.add("Anurag");
      names.add(null);
      names.add("Amit");

      String concat = "";
      int maxLength = 0;

      for (String name : names) {
         if (name != null) {
            concat += name;
            if (name.length() > maxLength) {
               maxLength = name.length();
            }
         }
      }

      System.out.println(concat + " | " + maxLength);
   }
}
```

---

3 filter() comparisons + reduce() Aggregate

```
import java.util.*;

public class FilterComparisonReduceAggregate {
   public static void main(String[] args) {

      List<Integer> salaries = new ArrayList<Integer>();
      salaries.add(20000);
      salaries.add(40000);
      salaries.add(60000);

      int sum = 0;
      int max = 0;

      for (Integer s : salaries) {
         if (s >= 30000) {
            sum += s;
            if (s > max) {
               max = s;
            }
         }
      }

      System.out.println(sum + " | " + max);
   }
}
```

---

4 filter() method call boolean + reduce() Aggregate

```
import java.util.*;

public class FilterMethodReduceAggregate {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {

      List<User> users = new ArrayList<User>();
      users.add(new User("Anurag", true));
      users.add(new User("Amit", true));
      users.add(new User("Ravi", false));

      int count = 0;
      String concat = "";

      for (User u : users) {
         if (u.isActive()) {
            count++;
            concat += u.name;
         }
      }

      System.out.println(count + " | " + concat);
   }
}
```

5 distinct() equals/hashCode + reduce() Aggregate

```
import java.util.*;

public class DistinctEqualsReduceAggregate {

   static class Person {
      String name;

      Person(String name) {
         this.name = name;
      }

      public boolean equals(Object o) {
         if (o instanceof Person) {
            Person p = (Person) o;
            return name.equals(p.name);
         }
         return false;
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> people = new ArrayList<Person>();
      people.add(new Person("Anurag"));
      people.add(new Person("Anurag"));
      people.add(new Person("Ravi"));

      Set<Person> unique = new HashSet<Person>(people);

      String concat = "";
      for (Person p : unique) {
         concat += p.name;
      }

      System.out.println(concat);
   }
}
```

6 distinct() immutable objects + reduce() Aggregate

```
import java.util.*;

public class DistinctImmutableReduceAggregate {
   public static void main(String[] args) {

      List<String> langs = new ArrayList<String>();
      langs.add("Java");
      langs.add("Java");
      langs.add("Go");

      Set<String> unique = new HashSet<String>(langs);

      String concat = "";
      int maxLen = 0;

      for (String s : unique) {
         concat += s;
         if (s.length() > maxLen) {
            maxLen = s.length();
         }
      }

      System.out.println(concat + " | " + maxLen);
   }
}
```

---

7 map() transform objects + reduce() Aggregate

```
import java.util.*;

public class MapTransformReduceAggregate {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
      names.add("anurag");
      names.add("amit");

      String concat = "";
      int maxLen = 0;

      for (String name : names) {
         String upper = name.toUpperCase();
         concat += upper;

         if (upper.length() > maxLen) {
            maxLen = upper.length();
         }
      }

      System.out.println(concat + " | " + maxLen);
   }
}
```

---

8 map() create new objects + reduce() Aggregate

```
import java.util.*;

public class MapNewObjectReduceAggregate {

   static class UserDTO {
      String name;
      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
      names.add("Anurag");
      names.add("Amit");

      String concat = "";

      for (String name : names) {
         UserDTO dto = new UserDTO(name);
         concat += dto.name;
      }

      System.out.println(concat);
   }
}
```

---

9 map() pure functions + reduce() Aggregate

```
import java.util.*;

public class MapPureFunctionReduceAggregate {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {

      List<Integer> nums = new ArrayList<Integer>();
      nums.add(2);
      nums.add(3);
      nums.add(4);

      int sum = 0;
      int max = 0;

      for (Integer n : nums) {
         int sq = square(n);
         sum += sq;

         if (sq > max) {
            max = sq;
         }
      }

      System.out.println(sum + " | " + max);
   }
}
```

---

10 flatMap() return Stream + reduce() Aggregate

```
import java.util.*;

public class FlatMapStreamReduceAggregate {
   public static void main(String[] args) {

      List<List<Integer>> lists = new ArrayList<List<Integer>>();

      List<Integer> l1 = new ArrayList<Integer>();
      l1.add(1);
      l1.add(2);

      List<Integer> l2 = new ArrayList<Integer>();
      l2.add(3);
      l2.add(4);

      lists.add(l1);
      lists.add(l2);

      int sum = 0;

      for (List<Integer> list : lists) {
         for (Integer n : list) {
            sum += n;
         }
      }

      System.out.println(sum);
   }
}
```

---

11 flatMap() split data + reduce() Aggregate

```
import java.util.*;

public class FlatMapSplitReduceAggregate {
   public static void main(String[] args) {

      List<String> lines = new ArrayList<String>();
      lines.add("Java Stream");
      lines.add("Flat Map");

      String concat = "";

      for (String line : lines) {
         String[] words = line.split(" ");
         for (String word : words) {
            concat += word;
         }
      }

      System.out.println(concat);
   }
}
```

---

12 flatMap() expand collections + reduce() Aggregate

```
import java.util.*;

public class FlatMapExpandReduceAggregate {
   public static void main(String[] args) {

      Map<String, List<Integer>> map = new HashMap<String, List<Integer>>();

      List<Integer> listA = new ArrayList<Integer>();
      listA.add(1);
      listA.add(2);

      List<Integer> listB = new ArrayList<Integer>();
      listB.add(3);

      map.put("A", listA);
      map.put("B", listB);

      int sum = 0;

      for (List<Integer> values : map.values()) {
         for (Integer n : values) {
            sum += n;
         }
      }

      System.out.println(sum);
   }
}
```

---

13 sorted() natural ordering + reduce() Aggregate

```
import java.util.*;

public class SortedNaturalReduceAggregate {
   public static void main(String[] args) {

      List<Integer> nums = new ArrayList<Integer>();
      nums.add(5);
      nums.add(1);
      nums.add(3);

      Collections.sort(nums);

      int max = 0;

      for (Integer n : nums) {
         if (n > max) {
            max = n;
         }
      }

      System.out.println(max);
   }
}
```

---

14 sorted() custom comparator + reduce() Aggregate

```
import java.util.*;

public class SortedCustomReduceAggregate {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
      names.add("Anurag");
      names.add("Ravi");
      names.add("Amit");

      Collections.sort(names, new Comparator<String>() {
         public int compare(String s1, String s2) {
            return s2.compareTo(s1);                    // reverse order
         }
      });

      String concat = "";

      for (String name : names) {
         concat += name;
      }

      System.out.println(concat);
   }
}
```

---

15 sorted() multi-field compare + reduce() Aggregate

```
import java.util.*;

public class SortedMultiFieldReduceAggregate {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> emps = new ArrayList<Employee>();
      emps.add(new Employee("Anurag", 30));
      emps.add(new Employee("Anurag", 25));
      emps.add(new Employee("Ravi", 35));

      Collections.sort(emps, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {
            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0) {
               return nameCompare;
            }
            return e1.age - e2.age;
         }
      });

      int maxAge = 0;
      for (Employee e : emps) {
         if (e.age > maxAge) {
            maxAge = e.age;
         }
      }

      System.out.println(maxAge);
   }
}
```

16 peek() logging + reduce() Aggregate

```
import java.util.*;

public class PeekLoggingReduceAggregate {
   public static void main(String[] args) {

      List<Integer> nums = new ArrayList<Integer>();
      nums.add(1);
      nums.add(2);
      nums.add(3);

      int sum = 0;

      for (Integer n : nums) {
         System.out.println("Peek: " + n);
         sum += n;
      }

      System.out.println(sum);
   }
}
```

---

17 peek() debugging + reduce() Aggregate

```
import java.util.*;

public class PeekDebugReduceAggregate {
   public static void main(String[] args) {

      List<Integer> nums = new ArrayList<Integer>();
      nums.add(10);
      nums.add(20);
      nums.add(30);

      int max = 0;

      for (Integer n : nums) {
         if (n == 20) {
            System.out.println("Debug hit");
         }

         if (n > max) {
            max = n;
         }
      }

      System.out.println(max);
   }
}
```

---

18 peek() metrics + reduce() Aggregate

```
import java.util.*;

public class PeekMetricsReduceAggregate {
   public static void main(String[] args) {

      List<Integer> nums = new ArrayList<Integer>();
      nums.add(5);
      nums.add(10);
      nums.add(15);

      int counter = 0;
      int sum = 0;

      for (Integer n : nums) {
         counter++;                                        // metric tracking
         sum += n;
      }

      System.out.println("Sum: " + sum);
      System.out.println("Processed: " + counter);
   }
}
```